Algorithm for file updates in Python

Project description

A health care company has tasked me to create an algorithm using Python. This algorithm needs to check an allowed IP addresses list of permitted employees and remove ones that have been added to the remove list. These removed IP addresses will not be able to sign into the restricted subnetwork.

Open the file that contains the allow list

To start this algorithm I open the file that contains the current allow list.

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement

with open(import_file, "r") as file:
```

As seen in the screenshot above, I assigned the file name and the list of removed IP addresses. Then I used a `with` statement alongside the `open()` method in order to open the file in read-only mode since I specified the "r" argument.

Read the file contents

Next, I read the file and displayed the files contents to check what it contained.

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Display `ip_addresses`

print(ip_addresses)
```

Using the `read()` method within the `with` statement allows access to the contents of the file so that I can start manipulating the allowed IP addresses.

Convert the string into a list

The contents of the file now needs to be converted into a list so that each element can be analyzed and changed accordingly.

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Display `ip_addresses`

print(ip_addresses)
```

In the above screenshot, I used the `split()` method to convert and reassign the `ip_addresses` variable to a list of IP addresses instead of one string full of them.

Iterate through the remove list

In order to check though and analyze the contents of the allowed IP addresses list, a for loop must be used to iterate through each element.

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Display `element` in every iteration

    print(element)
```

I named the loop variable `element` and I iterated through the elements of the `ip_addresses` list.

Remove IP addresses that are on the remove list

The list being iterated through now needs to be analyzed for if there are any matches with the removed IP addresses list. When there is a match, the IP address should be removed from the approved IP addresses list.

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Display `ip_addresses`

print(ip_addresses)
```

Within the `for` loop I use a conditional statement to check if the element is in the `remove_list`. If the element is in `remove_list`, the `remove()` method will remove the specified element from the `ip_addresses` list.

Update the file with the revised list of IP addresses

Finally the allowed IP addresses file needs to be reopened to rewrite the new list to it.

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:

  # Rewrite the file, replacing its contents with `ip_addresses`

  file.write(ip_addresses)
```

Before the file is reopened I used the `join()` method to convert the list back to a string by combining the elements and separating them with spaces. This is done so that allowed IP addresses are back in a format that can be put into the text file. Similar to the beginning of the algorithm, a `with` statement alongside the `open()` method is used to open the file but this time in write-only mode. The `write()` method is then used within the `with` statement to replace the text in the `allow_list.txt` file with the new string of IP addresses.

Summary

To complete the goal I was tasked with I used Python to read a given file and analyze it. The `open()` method is used to open up the file and read and write to it. The `split()` method is used to convert a string of IP addresses to a list for easy modifications. Since the IP addresses are converted to a list, the elements are easily iterated though and checked for matches with the `removed_list`. If a match is found the element will be removed from the `ip_addresses` list. Once the iteration through the list is complete the list will be converted back to a string using

the `join()` method and then using the `write()` method, the new string of IP addresses replaces the contents of the `allowed_list.txt` file.